Functions, packages, the tidyverse & objects

Day 3 - Wednesday, May 20, 2026

> ℹ Today: Wednesday May 20
>
> **Before class**
>
> - **Perusall Assignment**: *Data Computing* §3.1–3.5 (Click here).
> - **HW 0: Setup confirmation** (Click here).
>
> **Plan for today**
>
> - Recap & reading R code
> - Functions
> - Packages and the tidyverse
> - Histograms in ggplot2
> - In-class activity
>
> **Helpful links**
>
> - Syllabus · AI policy · Schedule · *Data Computing* §3

Quick recap from yesterday

Yesterday we covered:

- The four panes of RStudio (Source, Console, Environment, Output)
- R as a calculator
- Data types (numeric, integer, character, logical)
- Data structures (vectors, lists, matrices, data frames, arrays)
- Creating objects with `<-`

Today we'll build on that to start *reading* and *writing* real R code.

Reading R code

Yesterday's textbook reading made one key point:

> Before you can write R code, you need to be able to **read** it.

Most people who write code start by **copying and modifying** existing code.

To do that well, you need to recognize a few key patterns and know what each piece of an expression is doing.

Five kinds of objects in R code

When you look at a line of R code, you'll mostly see these five things:

1. **Data tables** - start with a capital letter (e.g., `BabyNames`)
2. **Functions** - always followed by an open parenthesis (e.g., `filter()`, `mean()`)
3. **Arguments** - what's *inside* the parentheses
4. **Variables** - the columns inside a data table (e.g., `name`, `sex`)
5. **Constants** - single values like `"Arjun"` or `1998`

> Note
>
> The capital/lowercase convention is a **style choice** that is followed by the *Data Computing* textbook; but it is not strictly enforced by R.

Spot the parts

```
BabyNames %>%
  filter(name == "Arjun") %>%
  summarise(total = sum(count))
```

Try to identify:

- **Data table?**
- **Functions?**
- **Variables?**
- **Constant?**
- **Named argument?**

This single command counts the total number of babies named "Arjun" in the data: **5,578**.

Spot the parts

```
BabyNames %>%
  filter(name == "Arjun") %>%
  summarise(total = sum(count))
```

Try to identify:

- **Data table?** `BabyNames` (starts with capital, no parentheses after it)
- **Functions?** `filter`, `summarise`, `sum` (each followed by an open parenthesis)
- **Variables?** `name`, `count` (lowercase, inside parentheses)
- **Constant?** `"Arjun"` (in quotes)
- **Named argument?** `total = sum(count)` (single `=` inside the function call)

This single command counts the total number of babies named "Arjun" in the data: **5,578**.

Objects, revisited

An **object** is anything in R that has a name you can refer to later.

```
age_cutoff <- 21
my_numbers <- c(3, 7, 12, 4)
greeting   <- "hello, world"
```

Once an object exists, you can use its name anywhere you'd otherwise type the value:

```
age_cutoff + 5
```

```
[1] 26
```

```
mean(my_numbers)
```

```
[1] 6.5
```

Why bother naming things?

Compare:

```r
# Hard to update, hard to read:
filter(students, age >= 21 & gpa >= 3.5 & credits >= 60)
```

vs.

```r
# Easy to read and change:
age_min     <- 21
gpa_min     <- 3.5
credit_min  <- 60

filter(students,
       age     >= age_min,
       gpa     >= gpa_min,
       credits >= credit_min)
```

Good names make your code **readable** and **easy to change later**.

Example of naming conventions from *Data Computing*

Table 1: Naming conventions from *Data Computing*.

Object type	Convention	Example
Data tables	Capitalized	`BabyNames`, `Penguins`
Variables / columns	lowercase	`name`, `body_mass_g`
Constants you create	lowercase, snake_case	`age_cutoff`, `gpa_min`

> ⚠ Warning
>
> Object names **must** start with a letter and can only contain letters, numbers, `_`, and `.`. No spaces, no special characters, no starting with a number. This is a **requirement**, not a convention.

Functions

What is a function?

A **function** is a reusable chunk of code that takes some input, does something with it, and gives you back an output.

You've already used many functions:

```r
sqrt(16)
```

```
[1] 4
```

```r
mean(c(1, 2, 3, 4, 5))
```

```
[1] 3
```

```r
c(1, 8, 4)
```

```
[1] 1 8 4
```

Every function call has the same three key elements: a **name**, followed by **parentheses**, with **arguments** inside.

Anatomy of a function call

```r
function_name(argument1, argument2, name = value, ...)
```

- The **name** tells R *which* function to use.
- The **arguments** tell the function *what to work on* and *how*.
- Arguments are separated by commas.
- Some arguments are **named** (`name = value`); others are matched by position.

```r
# Positional: round(x, digits)
round(3.14159, 2)
```

```
[1] 3.14
```

```r
# Named: order doesn't matter
round(digits = 2, x = 3.14159)
```

```
[1] 3.14
```

Named vs. positional arguments

```r
# Both of these work the same way:
seq(1, 10, 2)
```

```
[1] 1 3 5 7 9
```

```r
seq(from = 1, to = 10, by = 2)
```

```
[1] 1 3 5 7 9
```

> A good habit
>
> - Use **positional** arguments for the obvious ones (the data, the first input).
> - Use **named** arguments for everything else.
>
> Your future self will thank you when reading the code six weeks from now.

Getting help with a function

If you don't remember what arguments a function takes:

```r
?mean        # opens the help page
help(mean)   # same thing
??mean       # broader search if you're unsure of the name
```

Every help page has the same sections: **Description**, **Usage**, **Arguments**, **Value**, **Examples**.

Functions return values

Every function gives you back a **value**. You can store that value with `<-`:

```r
x <- sqrt(16)
x
```

```
[1] 4
```

```r
m <- mean(c(2, 4, 6, 8))
m
```

```
[1] 5
```

If you don't assign the result, R just prints it to the console and it's gone. The result will not be stored anywhere for you to access in the future.

> ⚠ Example
>
> ```r
> mean(c(2, 4, 6, 8))   # prints 5, but doesn't save it anywhere
> ```
>
> If you want to use that 5 later, you need `m <- mean(c(2, 4, 6, 8))`.

Functions inside functions

The output of one function can become the input to another:

```r
sqrt(mean(c(1, 4, 9, 16, 25)))
```

```
[1] 3.316625
```

R works **inside-out**:

1. First `c(1, 4, 9, 16, 25)` makes a vector.
2. Then `mean(...)` of that vector is `11`.
3. Then `sqrt(11)` is `3.317`.

This nesting can get unreadable fast. Later on, we'll see a cleaner way (the **pipe**).

Packages

What is a package?

Base R has a lot of features, but many useful tools live in **packages**.

A **package** is a bundle of:

- functions

- data sets
- documentation

Packages are written by someone in the R community and made available for everyone to use.

There are over **23,000** packages on CRAN (the main R package repository).

Installing vs. loading

There are **two steps** to using a package.

> **!** Install once, load every session
>
> - **Install** with `install.packages("packagename")` - do this **once** per computer.
> - **Load** with `library(packagename)` - do this **every time** you start a new R session.

```r
# Run this once, in the Console (NOT in your script):
install.packages("tidyverse")

# Run this at the top of every script or .qmd that uses it:
library(tidyverse)
```

Quotation marks matter: `install.packages("tidyverse")` (string), but `library(tidyverse)` (object).

Where do I install packages?

> Run `install.packages()` in the **Console**, not in your script.
>
> You only need to install once, so it doesn't belong in a script that runs every time.

If your script needs a package, just put `library(packagename)` at the top.

What happens when you load a package?

```r
library(tidyverse)
```

```
 Attaching core tidyverse packages        tidyverse 2.0.0
dplyr     1.1.4      readr     2.1.5
forcats   1.0.0      stringr   1.5.1
ggplot2   3.5.1      tibble    3.2.1
lubridate 1.9.4      tidyr     1.3.1
purrr     1.0.4
 Conflicts                  tidyverse_conflicts()
dplyr::filter() masks stats::filter()
dplyr::lag()    masks stats::lag()
```

These messages are **fine**. They mean: "Here are the packages I just loaded, and here are the function names that now point to a new place."

The tidyverse

What is the tidyverse?

The **tidyverse** is a *collection* of R packages that share a common philosophy and work well together.

When you run `library(tidyverse)`, you load all of them at once:

- **`ggplot2`** - making plots
- **`dplyr`** - manipulating data tables
- **`tidyr`** - reshaping data
- **`readr`** - reading data files
- **`tibble`** - a nicer kind of data frame
- **`stringr`** - working with text
- **`forcats`** - working with categorical data
- **`purrr`** - applying functions across collections

We will use almost all of them this semester.

The tidyverse hex stickers

Each tidyverse package has its own hex sticker - a fun tradition in the R community.

Why use the tidyverse?

Two reasons:

1. **Consistency.** Tidyverse functions are designed to work the same way:
    - First argument is always the data.
    - Output is always a data frame (a “tibble”).
    - You can chain them together cleanly (more on that later).
2. **Readability.** Tidyverse code reads almost like English.

We’ll spend most of the semester learning to *think* in this style.

A taste of what’s coming

Don’t worry about understanding every line - just notice how **readable** this is:

```
library(palmerpenguins)

penguins |>
  filter(!is.na(body_mass_g)) |>
  group_by(species) |>
```

```r
  summarize(
    n           = n(),
    avg_mass_g  = mean(body_mass_g)
  )
```

```
# A tibble: 3 x 3
  species       n avg_mass_g
  <fct>     <int>      <dbl>
1 Adelie      151      3701.
2 Chinstrap    68      3733.
3 Gentoo      123      5076.
```

This is the **dplyr** package in action. We'll cover it in detail later this week.

ggplot2: a grammar of graphics

`ggplot2` (part of the tidyverse) is the most popular plotting package in R.

It's built on the idea of a **grammar**: you build up a plot by adding layers.

Every `ggplot2` plot has at least three pieces:

1. **Data** - a data frame.
2. **Aesthetics** (`aes(...)`) - which variables map to which parts of the plot (x, y, color, fill, etc.).
3. **Geometry** (`geom_*()`) - what *kind* of plot (points, bars, lines, histograms…).

Histograms

A **histogram** shows the *distribution* of a single numeric variable by bucketing values into bins and counting how many fall in each bin.

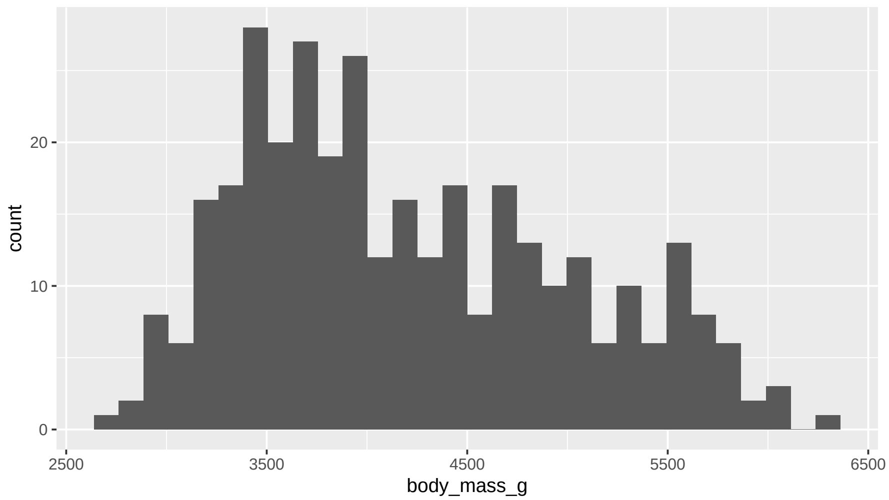
It's the right tool when you want to ask:

- What values are typical?
- Are there outliers?
- Is the distribution symmetric? Skewed? Bimodal?

In `ggplot2`, you make a histogram with `geom_histogram()`.

A basic histogram

```r
ggplot(penguins, aes(x = body_mass_g)) +
  geom_histogram()
```



Reading that code

```r
ggplot(penguins, aes(x = body_mass_g)) +
  geom_histogram()
```

Breaking it down:

- `ggplot(penguins, ...)`: start a plot using the `penguins` data.
- `aes(x = body_mass_g)`: map the variable `body_mass_g` to the **x-axis**.
- `+ geom_histogram()`: add a histogram layer.

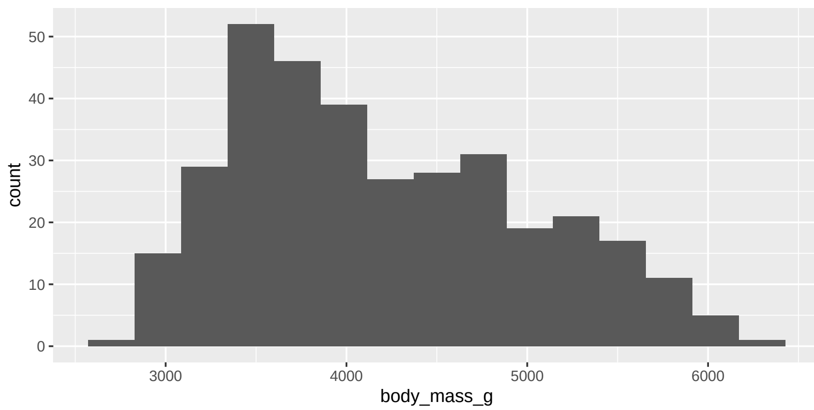
> **Note**
>
> In `ggplot2`, you add layers with `+`, not with `%>%` or `|>`.

Controlling the bins

R will warn you: *"stat_bin() using bins = 30. Pick better value with `binwidth`."*

You can control the number of bins two ways:

```r
ggplot(penguins, aes(x = body_mass_g)) +
  geom_histogram(bins = 15)
```




```r
# Or specify the width of each bin:
ggplot(penguins, aes(x = body_mass_g)) +
  geom_histogram(binwidth = 250)
```

Why the bin choice matters

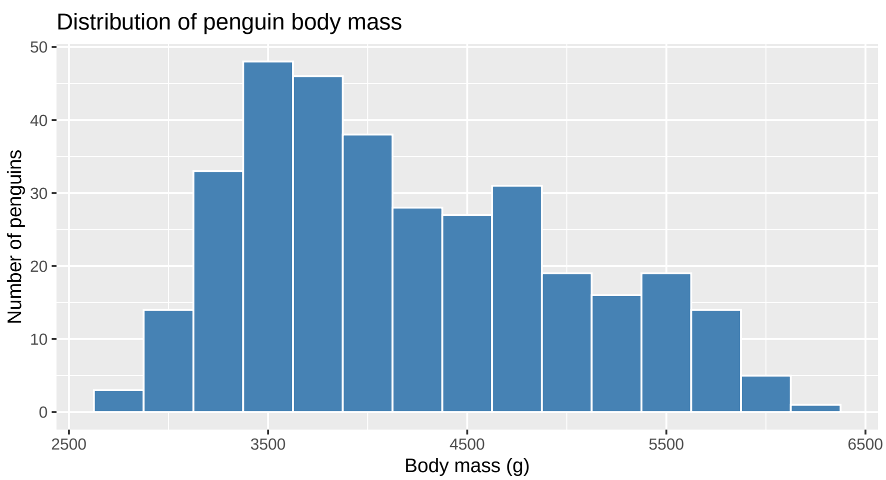
> ⚠ Too few bins
>
> You lose detail and the shape gets oversmoothed.
>
> **Too many bins**
>
> You see noise and the shape gets jagged and hard to read.

There's no single "right" answer. **Try a few**, and pick what tells the clearest story about your data.

Making it nicer

Add fill, color, and labels:

```
ggplot(penguins, aes(x = body_mass_g)) +
  geom_histogram(binwidth = 250, fill = "steelblue", color = "white") +
  labs(
    title = "Distribution of penguin body mass",
    x = "Body mass (g)",
    y = "Number of penguins"
  )
```



Anatomy of that command

Table 2: Anatomy of a basic histogram in ggplot2.

Piece	What it does
`ggplot(penguins, aes(x = body_mass_g))`	Use the penguins data; put `body_mass_g` on x
`geom_histogram(...)`	Add the histogram layer
`binwidth = 250`	Each bar is 250 g wide
`fill = "steelblue"`	Bar color
`color = "white"`	Outline color
`labs(...)`	Title and axis labels

> **Tip**
>
> You can run `?geom_histogram` in the Console to see *every* option.

Activity

Activity: your first histogram

You'll work on this individually or with a neighbor for 20 minutes. We'll regroup at the end.

Goal: Make a histogram of a real dataset and tweak it.

Setup:

1. Download the `day3-activity.R` starter file from Canvas.
2. Open RStudio.
3. Go to `File > Open File...` and open the script you downloaded.

Step 1 - Load packages

At the top of your script, type and run:

```
library(tidyverse)
library(palmerpenguins)
```

If `palmerpenguins` isn't installed, run `install.packages("palmerpenguins")` in the **Console** first.

Step 2 - Look at the data

```
# Pick ONE of these to peek at the data:
glimpse(penguins)
View(penguins)
?penguins
```

What variables are in this data set? What does each row represent?

Step 3 - Make a default histogram

Pick a numeric variable from `penguins` (e.g., `flipper_length_mm`, `bill_length_mm`, `bill_depth_mm`).

```r
ggplot(penguins, aes(x = flipper_length_mm)) +
  geom_histogram()
```

Step 4 - Tweak your histogram

Make at least **three** changes to your histogram:

- Try a different `binwidth` or `bins` value.
- Change the `fill` and `color` of the bars.
- Add a title and axis labels with `labs(...)`.

Step 5 - Comment

Open a comment block in your script and write 2–3 sentences answering:

```r
# What does the shape of your histogram tell you about the variable?
# Did changing the binwidth change the story?
```

To turn in:

Save your script (`Cmd/Ctrl + S`). Upload the `.R` file to the Canvas assignment page.

> Stuck?
>
> - Read the error message - it's usually a typo, missing `+`, or unmatched `(`.
> - Ask a neighbor.
> - Wave me down.

Activity debrief

Some things to look out for:

- **Forgot `+` between layers**: R thinks the line is finished.
- **Used `=` instead of `<-`**: works in some places, but breaks elsewhere.
- **Typo in a variable name**: R doesn't guess. The spelling and capitalization must match exactly.
- **Package not loaded**: `Error: could not find function "ggplot"`: run `library(tidyverse)`.

Wrap-up: what we covered today

- Recognize the five kinds of objects in R code.
- Read and call functions, including named and positional arguments.
- Install and load **packages**.
- Recognize the **tidyverse** as a coherent collection of packages.
- Build a **histogram** in `ggplot2` from scratch.

Wrap-up & looking ahead

Tomorrow: Quarto basics, writing reproducible documents.

Upcoming Deadlines

- **Wed 5/20, 11:59 p.m.** - HW 0: Setup
- **Thurs 5/21, before class** - DC §4.1–4.7
- **Fri 5/22, in class** - Quiz 1
  - Quiz 1 Practice
- **Fri 5/22, 11:59 p.m.** - HW 1: Basic R & Plots

Reach out

Stuck? Office hours, or email me (cgc5478@psu.edu) or Xinyue (xpw5228@psu.edu).

Reference

Syllabus · AI policy · Schedule · *Data Computing*